



TECHNICAL PAPER | TP-007 | VERSION 4.0

Natural/Adabas Modernization

Transforming Integrated Legacy Application and Data Architectures
for Relational and Cloud Platforms

Technology with Purpose. Opportunity with Impact.

BeyondX LLC | September 2026 | Version 4.0

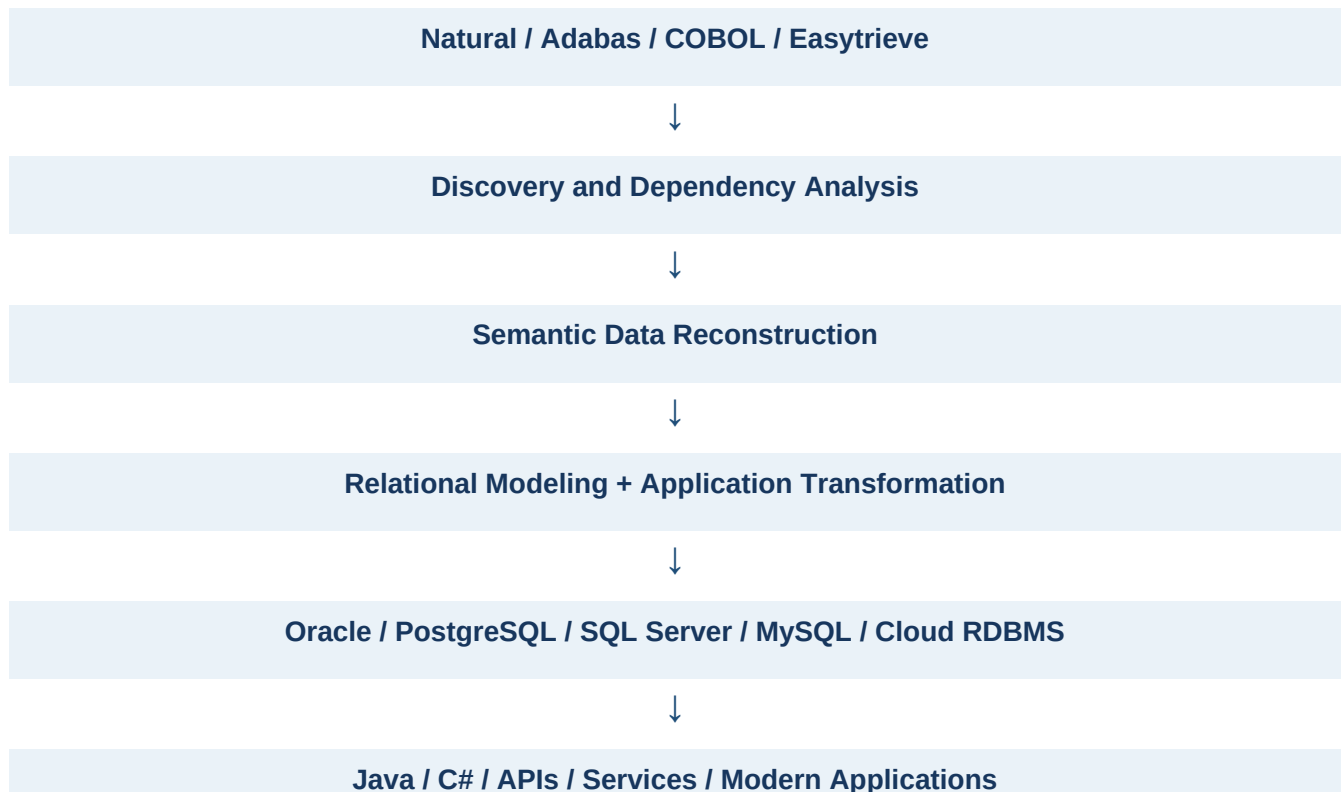
Executive Summary

Natural and Adabas have supported mission-critical enterprise applications for decades. Their longevity reflects an architecture engineered for efficient transaction processing, compact data representation, and close integration between application logic and database access.

Modernizing these environments is not simply a matter of converting Natural programs into Java or C#, nor is it accomplished by translating Adabas files directly into relational tables. Natural and Adabas form an integrated application and data ecosystem. In many enterprises, that ecosystem also includes COBOL programs, I/O copybooks, batch interfaces, shared record layouts, and downstream processes.

Natural applications may depend on Data Definition Modules (DDMs), Adabas file structures, descriptors, Internal Sequence Numbers (ISNs), Multiple Value fields (MU), Periodic Groups (PE), transaction behavior, record-access patterns, and other characteristics of the legacy environment. Successful modernization therefore requires understanding the complete dependency network before determining how it should be represented on a relational or cloud platform.

A typical transformation path:



CORE PRINCIPLE

The objective is not to reproduce every legacy structure literally. The objective is to preserve required business behavior and data integrity while deliberately transforming structures and

dependencies that no longer belong in the target architecture.

1. Understanding the Natural/Adabas Architecture

Adabas is fundamentally different from a conventional relational database management system. Its data structures and access paths were designed for a different application and processing model. The following mappings are conceptual rather than mechanical.

Adabas / Natural Concept	Approximate Relational or Modern Concept
Database	Database
File	Table or related set of tables - not necessarily one-to-one
Record	Row or related rows
Field	Column
ISN	Database-assigned record identifier / legacy identity
Descriptor	Indexed access path
Superdescriptor / Subdescriptor	Composite or derived indexed access path
MU field	Repeating values / child relation / alternative structured representation
PE group	Repeating structured group / child relation
Natural DDM	Application-oriented logical view of data

An Adabas file should not automatically become one relational table, and an Adabas descriptor should not automatically become one relational index. The target architecture should be derived from business semantics, application behavior, integrity requirements, workload characteristics, and future operational needs.

2. Adabas Data Structures

2.1 Files and Records

Adabas organizes information into files containing records. A record can contain repeating and grouped structures, allowing information that might require several related tables in an RDBMS to exist within one logical record.

ENGINEERING OBSERVATION

The physical boundaries of the legacy database do not necessarily represent the correct boundaries of the target relational model.

2.2 Internal Sequence Numbers (ISNs)

Adabas assigns an Internal Sequence Number (ISN) to identify records within a file. Natural applications may use ISNs for efficient direct access or retain an ISN so a record can later be retrieved or processed. During modernization, the ISN must therefore be treated as an application dependency as well as a database characteristic.

- Preserve the ISN temporarily when transition or reconciliation requires it.
- Retain it as a legacy identifier when historical traceability is valuable.
- Map it to a target surrogate key where appropriate.
- Use an existing business key when that key is authoritative and stable.
- Eliminate the ISN dependency only after dependent application behavior has been transformed and validated.

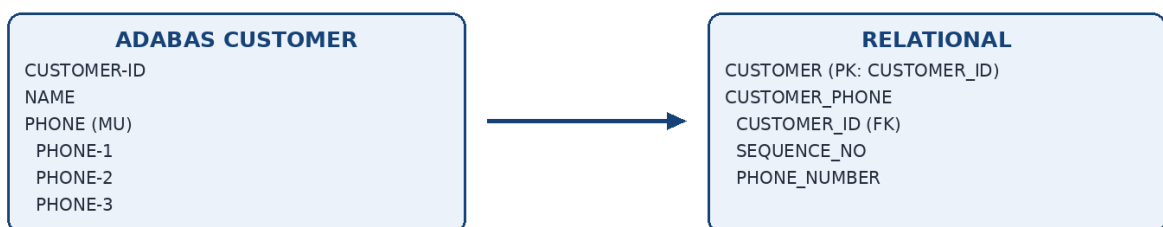
2.3 Multiple Value Fields (MU)

A Multiple Value field allows multiple occurrences of a value within an Adabas record. A relational implementation commonly externalizes these occurrences into a child relation, but the target design must preserve any business meaning attached to occurrence order, missing occurrences, update behavior, or downstream interfaces.

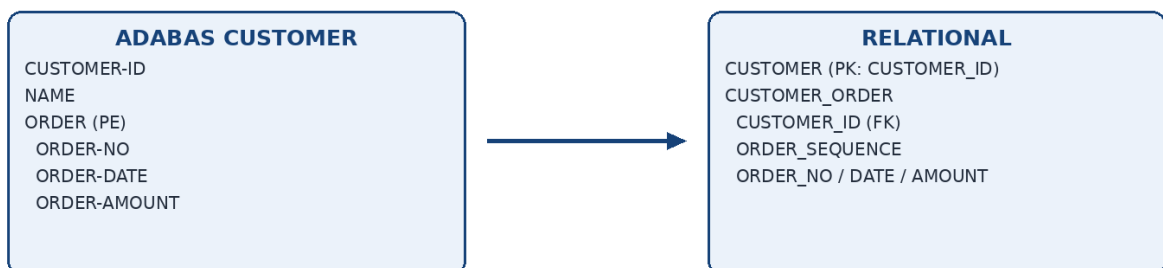
Adabas Repeating Structures to Relational Design

Preserve business semantics while transforming physical representation

Multiple Value (MU) Example



Periodic Group (PE) Example



2.4 Periodic Groups (PE)

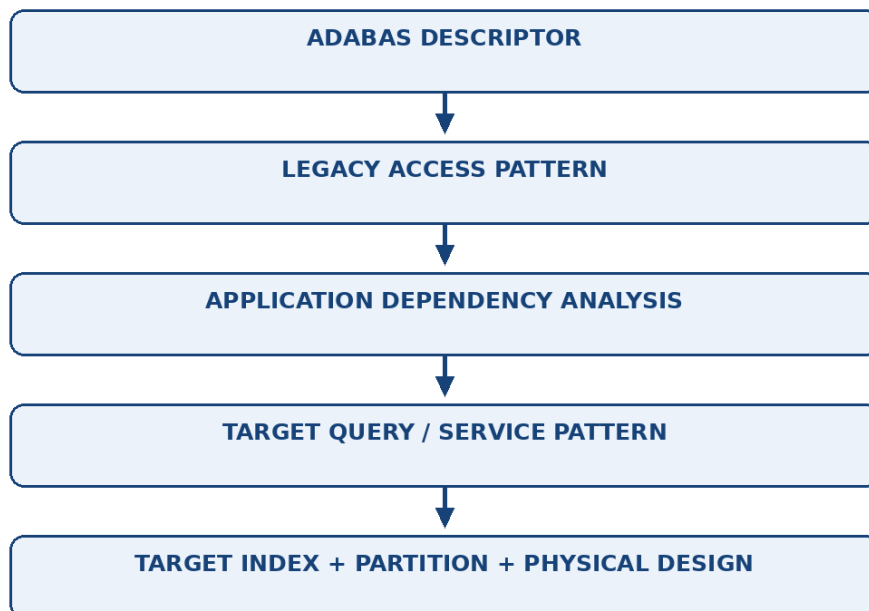
Periodic Groups allow repeating groups of related fields within a record. A PE may look like a nested table, but its occurrence semantics are part of the application contract and must be understood before normalization.

The transformation is not only structural. Natural code may depend on occurrence numbers, ordering, array processing, or assumptions about how the repeating group resides within the parent record.

3. Descriptors and Access Paths

Adabas descriptors provide efficient access paths to data. Superdescriptors can support access through combinations of fields, while derived descriptor forms can support access using portions or transformations of fields. These structures are sometimes treated as direct equivalents of relational indexes during migration; that assumption can produce a target physical design optimized for the legacy implementation rather than the target workload.

Descriptor-to-Target Physical Design



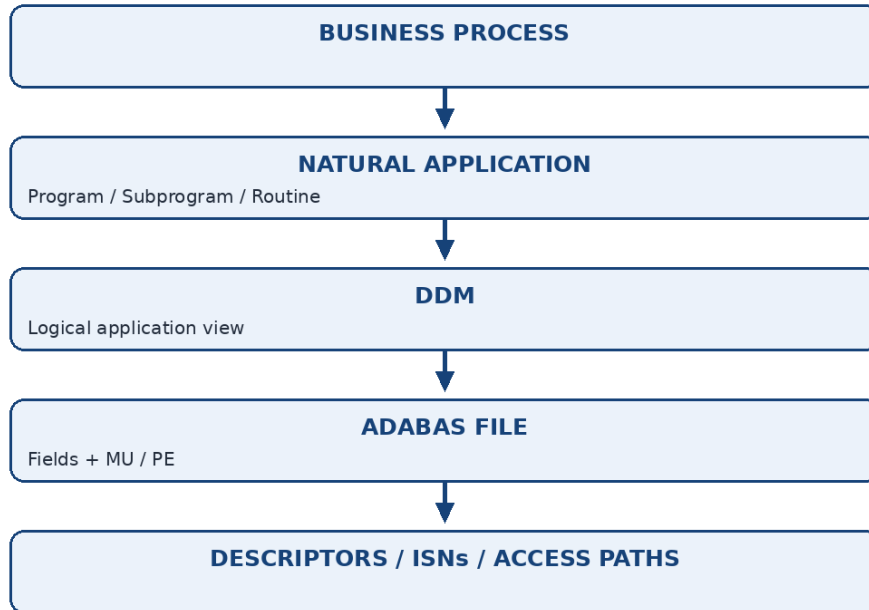
DESIGN PRINCIPLE

Do not convert indexes mechanically. Preserve the required access semantics, then engineer the target database for its optimizer, workload, concurrency model, and service boundaries.

4. Natural Application and Data Dependencies

Natural and Adabas were designed to operate closely together. Natural programs may contain assumptions about DDM definitions, file and field structures, descriptors, ISNs, MU/PE occurrences, field formats, transaction boundaries, shared routines, and database access behavior.

Natural Application and Data Dependency Chain

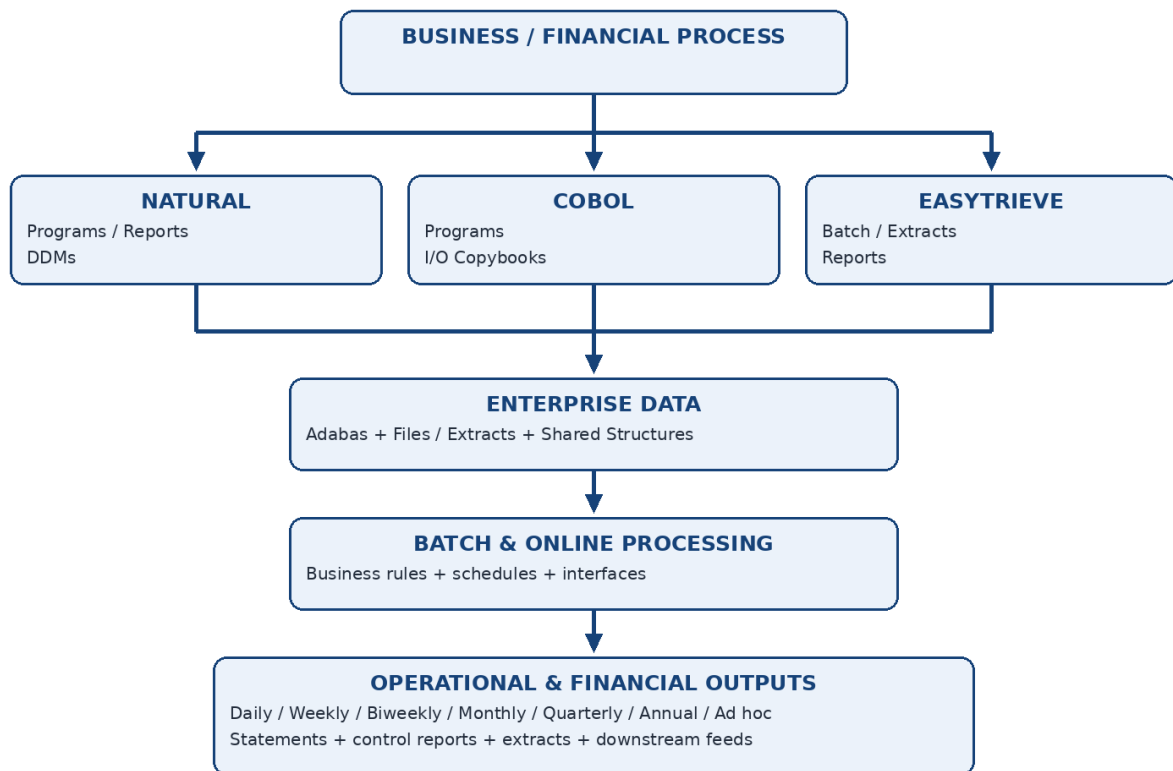


Changing the database without understanding this chain can produce a relational schema that is structurally valid but operationally incompatible with the application.

5. Mixed Natural, Adabas, COBOL, Easytrieve, and Copybook Environments

Natural/Adabas environments do not necessarily exist as self-contained stacks. In long-lived enterprise systems, Natural applications may coexist with COBOL and Easytrieve programs, I/O copybooks, batch jobs, extracts, and recurring reports that access, exchange, transform, or interpret the same business data. Modernization must therefore discover dependencies beyond Natural programs and Adabas database structures.

Mixed Legacy Application and Reporting Ecosystem



5.1 I/O Copybooks as Data Architecture Artifacts

An I/O copybook may define the physical or logical structure of information exchanged between programs, files, interfaces, batch processes, or database-related components. Copybooks should therefore be inventoried as part of data discovery, not treated solely as source-code artifacts.

- COBOL programs and copybooks
- Shared record layouts and I/O structures
- Batch interfaces and file dependencies
- Program-to-program interfaces
- Transformations between Natural, COBOL, Easytrieve, report outputs, and persisted representations

5.2 One Business Entity, Multiple Representations

The same business information may be represented in a Natural DDM, an Adabas definition, and one or more COBOL copybooks. Definitions that look equivalent should not automatically be assumed to have identical semantics. Over decades, differences can emerge in lengths, numeric representation, padding, signs, encoding, date formats, defaults, redefinitions, and application interpretation.

EXPERIENCE-BASED LESSON

Enterprise modernization experience in a client financial application environment reinforced

that application discovery and data discovery cannot be performed independently. Natural, Adabas, COBOL, and I/O copybook dependencies must be reconciled as one business dependency network.

5.3 Natural, Easytrieve, and Financial / Operational Reporting

In mature Natural/Adabas environments, reports are part of the application architecture. Natural and Easytrieve programs may generate financial statements, operational reports, control reports, extracts, and scheduled outputs. These artifacts can embody business rules that are not obvious from the database schema alone.

A modernization inventory should identify each recurring report and connect it to the programs, DDMs, Adabas files, copybooks, Easytrieve definitions, batch schedules, calculations, and downstream consumers that produce or depend on it.

Inventory Element	Discovery Requirement
Report / Statement	Business purpose and owner
Frequency	Daily, weekly, biweekly, monthly, quarterly, annual, ad hoc
Generating technology	Natural, Easytrieve, COBOL, or downstream tool
Source dependencies	DDMs, Adabas files, copybooks, extracts, files
Business logic	Selection criteria, fiscal periods, calculations, totals, subtotals, rounding
Output / distribution	Spool, print, file, PDF, extract, downstream interface
Validation	Counts, totals, balances, grouping, sorting, exceptions, formatting
Retention / audit	Historical retention, reconciliation, audit or regulatory use

ENGINEERING LESSON: A report is not merely an output artifact. In a mature Natural/Adabas environment, a report can embody business rules, calculations, fiscal-period logic, sorting, grouping, reconciliation rules, and audit expectations. Report discovery must therefore be part of application and data discovery.

6. Transactions, Data Representation, and Redefinition

6.1 Transaction Boundaries

Natural applications define logical units of work using transaction-processing constructs such as END TRANSACTION and BACKOUT TRANSACTION. When applications are transformed to Java, C#, services, or another modern architecture, these boundaries must be identified explicitly. A

statement-by-statement conversion can unintentionally alter transaction duration, lock duration, commit frequency, rollback behavior, concurrency, failure recovery, or restart behavior.

6.2 Implicit Conversions and Legacy Representations

Legacy applications may depend on implicit conversion, padding, missing-value conventions, character encodings, and historical date or numeric representations. Modern relational databases and strongly typed application languages can expose inconsistencies that remained tolerated or hidden in the legacy environment.

6.3 Natural and Adabas Date Representation

Date handling deserves explicit discovery during Natural/Adabas modernization because the stored representation, Natural D-format semantics, display conventions, report formatting, and target SQL date types are related but not identical. Software AG documents NATDATE as Natural D-format data expressed as a count of days since the year-zero reference; the E(NATDATE) definition specifies the count from January 2 of year 0. Adabas supports NATDATE values from 1582-01-01 through 9999-12-31, while a Natural session can impose its own MAXYEAR setting.

The physical representation can vary by field format. Software AG examples show E(NATDATE) represented as packed, unpacked, binary, or fixed numeric data; for example, a four-byte packed field may be defined with DT=E(NATDATE). Natural D fields provide date semantics to the application, while the DTFORM profile parameter controls default presentation conventions such as International YYYY-MM-DD or USA MM/DD/YYYY. The presentation format must therefore not be mistaken for the persisted representation.

Discovery item	Legacy concern	Target validation
Natural D / Adabas NATDATE	Internal day-count semantics and valid range	Convert to an explicit target DATE and compare boundary/representative values
DTFORM / report masks	Display order and delimiters may differ from storage	Reproduce required report and statement presentation independently of storage
Packed or numeric representations	Raw values may require conversion routines outside Natural	Profile and convert before load; do not treat raw packed values as business dates
Historical/invalid/zero values	Legacy conventions may not map directly to an RDBMS DATE	Define exception, null/default, and remediation rules before migration

MODERNIZATION LESSON: Validate dates at three levels - stored value, Natural application semantics, and rendered report/financial-statement output. A target database can contain technically valid dates while still producing incorrect fiscal-period selection, sorting, aging, or statement presentation if legacy date behavior is not reconstructed.

6.4 REDEFINE and Semantic Interpretation

Natural REDEFINE constructs can allow the same underlying data area to be interpreted using a different structure. During modernization, such constructs should be understood semantically rather than translated syntactically. The key question is not simply how to convert a REDEFINE into Java or C#, but what business meaning the application derives from that representation and how that meaning should be made explicit in the target architecture.

7. Do Not Relationalize Too Early

A major modernization risk is designing the target relational schema before reconstructing the semantics of the legacy environment. A safer sequence is:

Discover -> Understand -> Reconstruct Semantics -> Map Dependencies -> Define Target Logical Model -> Design Relational Model -> Design Physical Structures -> Transform Applications -> Migrate -> Validate

Beginning directly with ADABAS FILE -> SQL TABLE can preserve the wrong abstractions. Modernization should distinguish structures that represent genuine business concepts from structures that exist because of historical implementation constraints.

8. Modernization Patterns

8.1 Rehost / Re-platform

Preserve substantial portions of the Natural/Adabas architecture while moving the operating environment to a supported modern platform. This can reduce immediate transformation scope but may retain significant legacy dependencies.

8.2 Data-First Modernization

Transform Adabas data into a relational platform while providing transitional mechanisms for applications that cannot be modernized simultaneously.

8.3 Automated or Assisted Refactoring

Use modernization tooling to assist in transforming Natural applications into languages such as Java or C# and Adabas structures into relational representations. Generated code and schemas still require engineering review, testing, optimization, and validation.

8.4 Incremental Domain Modernization

Identify bounded business capabilities and progressively migrate their applications and data, accepting temporary coexistence and synchronization requirements.

8.5 Application and Data Transformation

Redesign both application and data architectures for the target environment, maximizing the opportunity to eliminate legacy constraints while increasing transformation and validation scope.

8.6 Replace

Replace the legacy application with a new custom, commercial, SaaS, or packaged solution while migrating required historical and operational data.

9. Lessons from Migration to Relational Platforms

9.1 Normalization Has Performance Consequences

An Adabas record containing MU and PE structures may become several relational tables. A legacy access operation that obtained required information from one record can become multiple SQL operations or joins. Target indexing, partitioning, query design, caching, application access patterns, and transaction design must therefore be engineered and performance-tested.

9.2 Data Profiling Must Begin Early

Legacy production data can contain representations accumulated across many years of application evolution. The target platform may impose different constraints on numeric values, dates, nullability, character sets, field lengths, precision, scale, and referential relationships. Profiling and cleansing should begin during discovery rather than immediately before migration.

9.3 Concurrency Must Be Re-engineered

Legacy and relational platforms can have substantially different transaction, isolation, and locking behavior. Simply reproducing the legacy transaction flow can introduce unexpected contention or deadlocks. Concurrency behavior must be tested under representative workloads.

9.4 Application Behavior Is Part of the Data Architecture

The meaning of legacy data cannot always be derived from database structures alone. Natural programs may reveal how fields are interpreted, which access paths matter, how repeating structures are traversed, how records are related, when transactions commit, and how historical values are interpreted.

9.5 Validation Must Be Continuous

Validation should occur throughout extraction, transformation, loading, application conversion, integration testing, performance testing, and production transition. It should include structural validation, data validation, relationship validation, business-rule validation, application-output validation, and performance validation.

10. Migration Engineering Lifecycle

Stage	Engineering Focus
1. Discovery	Inventory Natural objects, DDMs, Adabas files, descriptors, COBOL programs, copybooks, interfaces, batch flows, and operational dependencies.
2. Profiling	Measure data quality, occurrences, edge cases, invalid

	representations, volume, skew, and access patterns.
3. Semantic Reconstruction	Determine business entities, relationships, transaction semantics, field meanings, and cross-technology representations.
4. Target Modeling	Define logical relational model, keys, constraints, normalization boundaries, indexes, partitions, APIs, and service ownership.
5. Transformation	Convert data and application logic; preserve traceability between source and target.
6. Migration / Synchronization	Execute bulk movement and, where required, controlled coexistence or change synchronization.
7. Validation	Reconcile data, business outputs, interfaces, transactions, performance, security, and recovery behavior.
8. Cutover / Fallback	Transition operational ownership with explicit rollback or fallback criteria.
9. Decommission	Retire legacy components only after dependencies, retention, audit, and operational requirements are satisfied.

11. Validation and Reconciliation

Record counts alone are insufficient for a Natural/Adabas migration. Validation should include:

- File/record and target row counts
- Field-level value reconciliation
- MU and PE occurrence reconciliation
- Null, blank, default, and missing-value semantics
- Numeric precision and scale
- Date/time interpretation, NATDATE conversion, and report-format parity
- Character encoding and special characters
- Key and relationship integrity
- Business totals and control totals
- Application-output comparison
- Legacy-to-target report reconciliation: counts, totals, subtotals, balances, fiscal periods, rounding, grouping, sorting, and exceptions
- Batch and interface reconciliation
- Representative performance and concurrency testing

VALIDATION PRINCIPLE

Validation is strongest when legacy and modernized outputs can be compared side-by-side using known business scenarios and production-like data characteristics.

12. Cutover, Coexistence, and Fallback

Natural/Adabas modernization often spans multiple applications and technologies. A big-bang cutover may not be appropriate. Where coexistence is required, teams must explicitly define the system of record, synchronization direction, conflict handling, transaction boundaries, reconciliation cadence, and exit criteria.

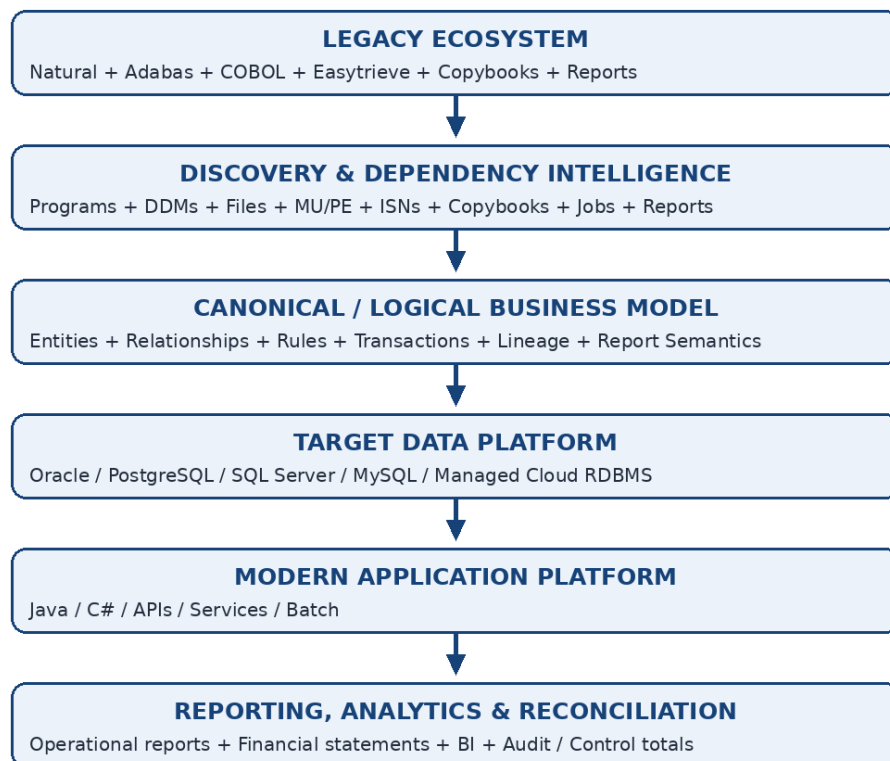
Fallback is not merely a technical switch. It requires a known recoverable state, compatible data, operational procedures, and clear decision criteria. The modernization plan should define how the organization returns safely to the prior operating state if required.

13. Target Architecture

A target architecture should decouple business capabilities from legacy physical structures while preserving the semantics needed for continuity and auditability.

Natural/Adabas Modernization - Target Architecture

Modernize the dependency network, not only the database

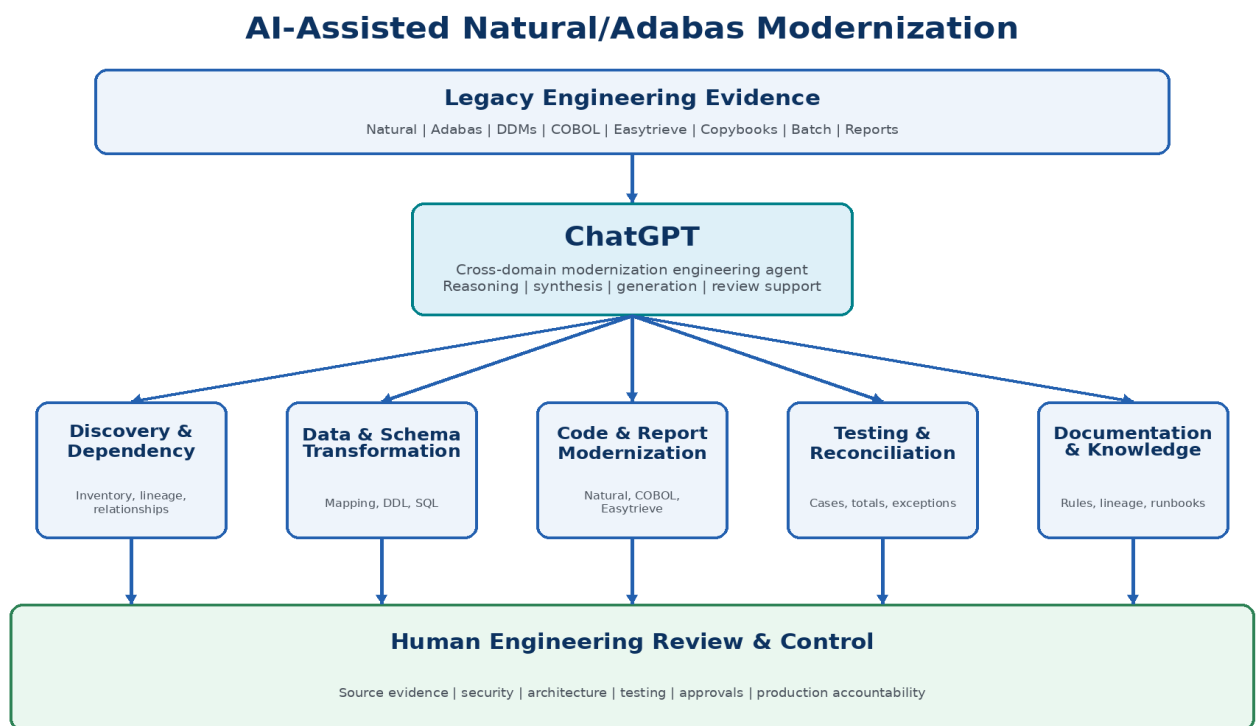


14. AI-Assisted Natural/Adabas Modernization

Modernization programs increasingly benefit from AI agents that can reason across legacy source code, data definitions, reports, batch artifacts, target schemas, test evidence, and documentation. In a Natural/Adabas estate, AI is most valuable when used as an engineering accelerator across the full modernization lifecycle - not merely as a code-conversion utility.

14.1 ChatGPT as a Modernization Engineering Agent

ChatGPT can serve as a collaborative engineering agent across discovery, dependency analysis, semantic reconstruction, target design, transformation, testing, reconciliation, documentation, and decommissioning analysis. Its horizontal role is especially useful when engineers must connect multiple technology generations - Natural, Adabas, DDMs, COBOL, Easytrieve, I/O copybooks, batch processing, reports, relational databases, APIs, cloud services, and modern application languages.

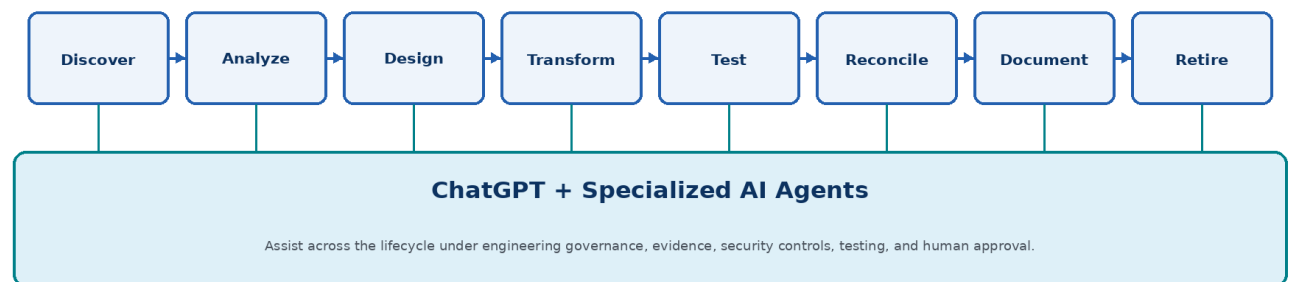


14.2 Where AI Agents Contribute

- **Discovery and inventory:** Analyze Natural source, DDMs, Adabas definitions, COBOL, copybooks, Easytrieve, batch artifacts, interfaces, and reports to accelerate inventories and identify areas requiring deeper investigation.
- **Dependency intelligence:** Correlate programs, DDMs, Adabas files, copybooks, jobs, extracts, and reports to help construct application-to-data-to-output lineage.
- **Semantic reconstruction:** Surface business meaning embedded in REDEFINE logic, MU/PE structures, descriptors, ISNs, date representations, implicit conversions, calculations, transaction behavior, and report rules.

- **Target architecture and data modeling:** Generate candidate relational structures, keys, relationships, MU/PE transformations, indexes, APIs, service boundaries, reporting designs, and cloud database patterns for engineering review.
- **Code and report modernization:** Assist Natural-to-Java/C# refactoring, COBOL and Easytrieve analysis, SQL generation, batch redesign, report conversion, API scaffolding, and repetitive transformation work.
- **Migration engineering:** Draft mapping specifications, DDL, conversion SQL, reconciliation queries, exception logic, runbooks, and fallback procedures.
- **Testing and reconciliation:** Generate boundary and regression cases, compare source and target results, investigate mismatches, and automate checks for counts, totals, balances, dates, reports, and business rules.
- **Documentation and knowledge recovery:** Transform legacy evidence and engineering decisions into data dictionaries, lineage, rule catalogs, interface specifications, architecture documentation, test evidence, and operational procedures.
- **Decommissioning analysis:** Identify residual references to Natural, Adabas, COBOL, Easytrieve, files, jobs, interfaces, and reports before retirement decisions are made.

14.3 AI-Assisted Engineering Lifecycle



AI agents can participate repeatedly across discovery, analysis, design, transformation, testing, reconciliation, documentation, and retirement preparation. The engineering team remains responsible for selecting authoritative source evidence, resolving ambiguity, validating generated artifacts, approving production changes, and determining whether modernization acceptance criteria have been met.

14.4 AI Governance, Security, and Human Engineering Control

For government, regulated, and security-sensitive environments, AI adoption must operate within the organization's approved security and governance boundary. Source code, production data, credentials, sensitive records, and controlled information should only be processed by AI services and environments authorized for that information. Modernization teams should define access controls, data-handling rules, prompt and artifact retention requirements, auditability, model/tool approval, and human review responsibilities before AI agents are introduced into engineering workflows.

- AI-generated mappings, code, SQL, test cases, and documentation should be treated as engineering candidates until validated against authoritative source evidence.

- Generated artifacts should be traceable to the source programs, data definitions, requirements, or test evidence that support them.
- Security scanning, code review, data validation, performance testing, and business acceptance remain required regardless of whether an artifact was human-written or AI-assisted.
- Production deployment, cutover, rollback, and legacy-retirement decisions remain accountable human and program decisions.

BEYOND X ENGINEERING PRINCIPLE

AI should not replace modernization engineering. It should amplify it. AI agents accelerate discovery, analysis, transformation, testing, reconciliation, and documentation; engineering judgment determines whether the result preserves business semantics, security requirements, operational behavior, and data integrity.

15. BeyondX Engineering Lessons

ENGINEERING LESSON 1

Natural/Adabas modernization should not begin by asking how an Adabas file can be converted into a relational table. It should begin by discovering how Natural applications, DDMs, record structures, MU and PE occurrences, descriptors, ISNs, transactions, COBOL programs, copybooks, and business processes depend upon that data.

ENGINEERING LESSON 2

The modernization unit should be the business dependency network - not an individual programming language, database, file, DDM, or copybook.

ENGINEERING LESSON 3

Tools can accelerate code conversion and schema transformation, but generated output does not prove semantic equivalence, transactional correctness, operational performance, or readiness to retire the legacy platform.

16. Modernization Success and Legacy Retirement

A successful Natural/Adabas modernization is not demonstrated simply by Natural code being converted to Java or C#, or by Adabas data being loaded into a relational database. Success requires demonstrating that required business behavior, data integrity, operational characteristics, reconciliation requirements, security controls, recovery processes, and downstream dependencies operate correctly on the target architecture.

Only after those dependencies have been addressed can the legacy environment be retired with confidence.

17. The BeyondX Modernization Principles

BEYOND X PRINCIPLE

Discovery is not a phase - it is a discipline.

BEYOND X PRINCIPLE

Validation is not a milestone - it is continuous.

BEYOND X PRINCIPLE

Tools accelerate modernization - but they do not replace engineering judgment.

BEYOND X PRINCIPLE

Modernization is successful only when the legacy system can be retired with confidence.

Modernization scope should explicitly include Natural, Adabas, COBOL, Easytrieve, I/O copybooks, batch schedules, extracts, and recurring reports when they participate in the same end-to-end business process.

AI agents can accelerate modernization - but they do not replace engineering judgment, evidence, security controls, testing, or accountable human decisions.

Conclusion

Natural/Adabas modernization is fundamentally a transformation of integrated application and data dependencies. Adabas files, MU and PE structures, descriptors, ISNs, Natural DDMs, transaction behavior, COBOL programs, copybooks, and interfaces collectively encode decades of business behavior. Treating any one of these artifacts in isolation can leave critical assumptions undiscovered.

The strongest modernization programs reconstruct those semantics first, design the target relational and application architecture second, and continuously validate business behavior throughout migration. This approach turns modernization from a syntax-conversion exercise into an engineering discipline capable of supporting confident legacy retirement.